Polaris Flow 使用者速查卡

v1.0 · 2026-10-19 · 面向工具链使用者：装上它 → 用它干活 → 配合它 → 别踩坑

① 装上它 / 照看它 polaris --help 仅列这些；hooks 运行时命令走 polaris-flow，不用记

命令	什么时候用	要点
`polaris init`	第一次用：装工作流 + Harness 环境	需访问 GitHub；不通时设 `POLARIS_GITHUB_MIRROR` 镜像，或用 `POLARIS_SUPERPOWERS_PATH` 指向已下载的本地目录
`polaris doctor`	装了没、环境对不对	诊断环境、schema 与 skill 安装状态，先跑它再排查
`polaris status`	现在有什么任务、进行到哪了	当前 change 与工作流状态
`polaris dashboard`	想看一眼全局进度	本地工作台，API 与界面同端口；`--api-only` 供开发联调
`polaris update`	升级：程序本体 / 工作流资产 / 第三方依赖	可只更一类，例 `update --only skills`；`--prune` 清陈旧产物；`--lang zh` 切语言
`polaris uninstall`	卸载	占位功能，尚未实现

② 命令对号入座 13 个命令 · 按场景选，右列是开工前你要备好的东西

你的场景	命令	它干什么	开工前你要有
想法模糊，要写成正式需求	`/psfl:prd`	需求基线 + 澄清纪要	一句话想法即可；已有文档可作附加材料
PRD 定稿了，能不能开工	`/psfl:prd:readiness`	五维打分：PASS / CONDITIONAL / FAIL	定稿终稿（FAIL 会打回，不让你带病开工）
有需求，做页面原型	`/psfl:prototype`	先出蓝图 → 你确认 → HTML 原型 → 归档	需求基线 / 定稿 PRD / 建设方案，任一可读
原型做好，把关一次	`/psfl:prototype:review`	只出评审报告，不改文件、不归档	原型 .html 路径（必需，缺了会停下来要）
改个小地方（文案 / 字段 / 参数）	`/psfl:coding:tweak`	最快通道，一次会话跑完	做什么 + 对哪个对象 + 至少 2 项细节
实现一个功能（多模块协作）	`/psfl:coding:normal`	常规档，含规格契约与任务拆分	同上
复杂 / 高风险（跨服务跨系统）	`/psfl:coding:sdd`	完整阶段链，带深度设计	同上
线上故障，用户在受影响	`/psfl:maintance:hotfix`	生产通道：先保现场，再改	现象 + 时间线 + 影响面 + 变更清单（会问你补齐）
测试提了缺陷	`/psfl:maintance:bugfix`	测试通道，收尾归档故障记录	复现步骤 / 实际结果 / 预期结果 / 环境版本
写「怎么算做完」的判据	`/psfl:testing:acceptance`	GWT 格式验收标准清单	需求条目 / 功能点清单
设计测试用例	`/psfl:testing:create-testcase`	测试计划 + 用例集	定稿 PRD 且含验收标准
用例集找漏测	`/psfl:testing:review-testcase`	只出评审报告，不改用例、不阻断	待评审用例集路径
看看代码写得怎么样	`/psfl:flow → M11`	五维系统化审查（可选 SonarQube）	指定代码范围
拿不准用哪个	`/psfl:flow`	总入口：问清意图，判档并路由	一句话目标即可

暂不可用（会明确告诉你缺什么，不静默跳过）：R01 编写用户需求、M12 重构代码。需求文档四步走：`/psfl:prd`（写）→ `/psfl:prd:readiness`（验）→ 开发三档（做）→ retro（复盘，可选）。

③ 拿不准？走总入口 /psfl:flow · 只做选择与路由，不替你决定

1 选类别：需求/原型/开发/测试/维护 → **2 选功能** → **3 给附加上下文（文件/目录/说明，可无）** → **加载对应技能**

开发类进入后判档：入口先粗判（只升不降），**最终档位在澄清阶段末尾定，定了就冻结**。快捷路径：首次输入直接给编号（如 C02）或功能名，跳过前两步。**中断了怎么办**：重新输入当前命令名即可续跑，系统自动恢复在途任务并转交对应阶段，不用手动换命令。

④ 三档流程与你的事 每个阶段都会停下来等你确认一次——这是设计如此，不是卡住

简单 tweak：澄清 ←跟你确认理解对不对 → 任务规划 ←计划给你过目 → 构建 → 验收 ←结果你验收 → 交付。一次会话跑完。

常规 normal：澄清 → 提案 ←方案你点头才动笔 → 任务规划 → 构建 → 验收 → 交付。

严格 sdd：澄清 → 提案 → 深度设计（可选，提案结束时问你要不要深化）→ 任务规划 → 构建 → 验收 → 交付。

修复 Bug：诊断与方案 ←方案确认后才动手（可换方案 / 可转常规档） → 实现与自验 → 关单归档。生产通道先保现场再改。

澄清末尾可能升档（不会悄悄降）；交付后想说复盘，直接提 retro。越权确认一律不算数，见 ⑥。

⑤ 5 个不要 & 何时换通道

✕ 最常犯的错

- ✕ 需求说不清就开工 → 至少「**做什么 + 对哪个对象 + 2 项细节**」
- ✕ 提示新开会话还硬续 → 丢上下文，**照做**
- ✕ 嫌确认多想跳过 → **便宜时确认**比改完 20 个文件返工划算
- ✕ 半路变复杂还硬扛 → 听提示**切 normal / sdd**
- ✕ 想让评估往低判 → 宁严勿松，**只升不降**

▸ 何时换通道

- ▸ 一眼就是小改动 → 直达 **/psfl:coding:tweak**
- ▸ 线上告警 → 直达 **/psfl:maintance:hotfix**，先保现场
- ▸ 根因不明 / 跨 3+ 模块 / 表结构变更 / API 破坏性改动 → **退回 /psfl:coding:normal**
- ▸ 只想评现成文档 / 原型 / 用例 → review 类**只出报告，不阻断流程**

⑥ 硬纪律（不可破 · 速记）

类别	规则
命令名	Claude Code / Trae 用 **:** 分隔；**Cursor 换成 -**（如 `/psfl-coding-normal`），词不变
判档	拿不准一律 `/psfl:flow`；复杂度**只升不降**；澄清末尾定档后冻结，不反复横跳
开工门槛	「做什么 + 对哪个对象 + ≥2 项细节」，缺了**不放行**——防产出和意图错位
确认	**明确整体确认才算数**；「差不多」「可以吧」、沉默一律不算
推进	每阶段**停一次等你确认**；中断了输入当前命令名即可续跑，不用换命令
评审类	`prototype:review` / `testing:review-testcase` / `prd:readiness` **只出报告**：不改文件、不推流程
发布	发布 / 灰度 / 回滚**始终由人执行**，工具只负责把该确认的问你

⑦ 标准开场模板（贴给 AI 即可）

```
用 /psfl:flow。需求：[一句话说清做什么 + 对哪个对象 + 关键细节]。
材料：[PRD 路径 / 设计稿 / 目标代码路径，没有就删掉这行]。
按流程走：先澄清 → 评估复杂度判档 → 把评分依据给我看 → 再开工。
```

跨平台提醒：Cursor 侧把命令里的 **:** 全换成 **-**。开发类没需求不放行——这是防止产出和你的意图错位。